

ローカルミニマムの罠

初期値しだいで答えが変わる

rkskmt

1

本編の振り返りと今回の問題

2

本編でやったこと

本編では $f(x) = x^2 - 4x + 3$ という **2次関数** で勾配降下法を学びました。
この関数は **谷が1つだけ** のシンプルな形でした。どこから始めても、必ず同じ最小値 $(x = 2)$ にたどり着きました。

① 本編の結論

- 勾配降下法：傾きを測り、逆方向に少し進む、を繰り返す
- 学習率が適切なら、谷の底にたどり着く
- スタート地点を変えても、**たどり着く場所は同じ** だった

3

今回の疑問

でも、こんな疑問が浮かびます：

「**谷が2つ以上ある関数だったら、どうなるの？**」

実際の機械学習の問題では、谷が1つだけということはほとんどありません。パラメータが多くなると、**たくさんの谷** が存在します。

今回は「谷が2つある関数」に勾配降下法を放り込んで、**何が起こるか** を体験しましょう。

4

谷が2つある関数を見てみよう

5

関数の定義

今回使う関数はこれです：

$$f(x) = 0.1x^4 - x^3 + 2x^2 + 1$$

本編と同じように、微分を手で計算しておきます。

1次微分（傾き）： $f'(x) = 0.4x^3 - 3x^2 + 4x$

2次微分（曲がり具合）： $f''(x) = 1.2x^2 - 6x + 4$

6

Pythonでの定義

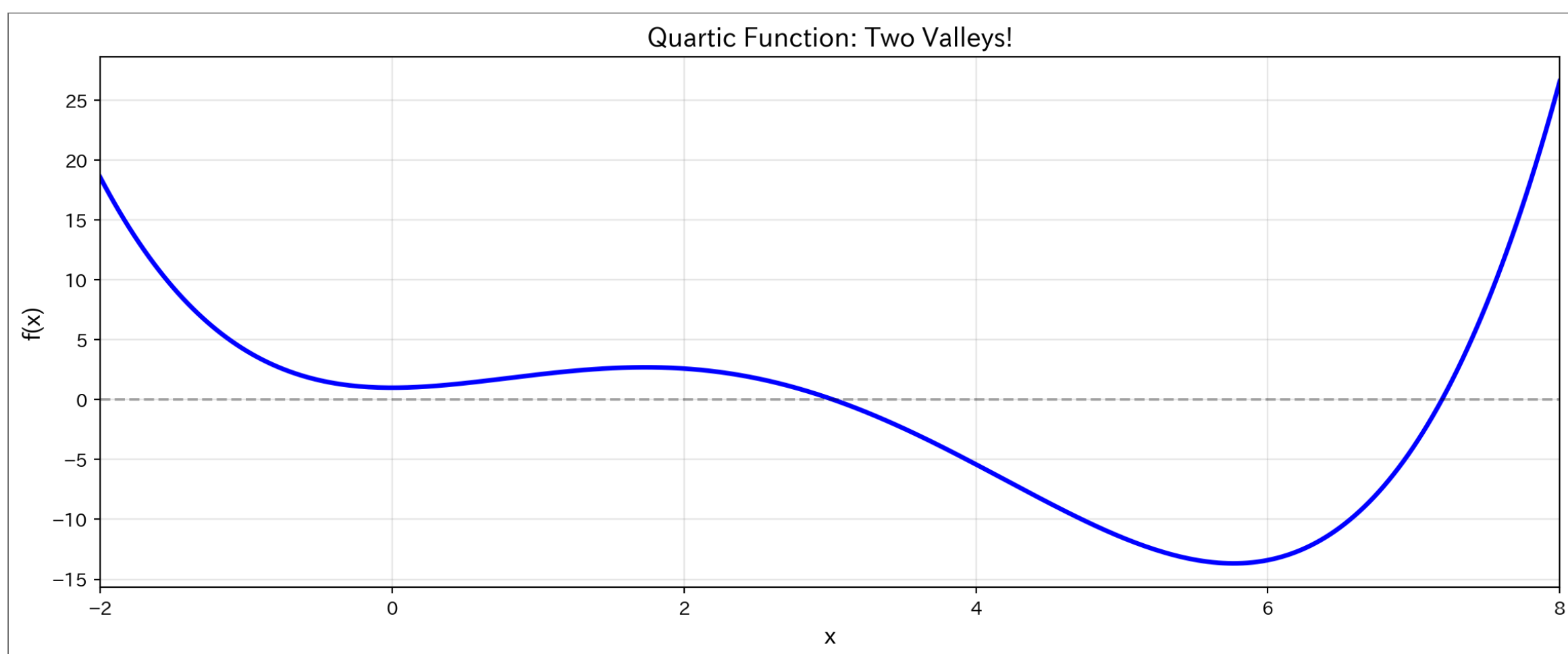
Code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 # 4次関数とその微分
6 def f_poly(x):
7     return 0.1*x**4 - x**3 + 2*x**2 + 1
8
9 def f_poly_prime(x):
10    return 0.4*x**3 - 3*x**2 + 4*x
11
12 def f_poly_double_prime(x):
13    return 1.2*x**2 - 6*x + 4
```

7

まずグラフを描いてみよう

どんな形の関数なのか、**まず目で見ること**が大切です。



① グラフを観察しよう

谷がいくつあるか、数えてみてください。

- **左の谷** ($x \approx 1$) 付近：浅い谷
- **右の谷** ($x \approx 6$) 付近：深い谷

本編の2次関数は谷が1つだけでしたが、今回は**2つ**あります。

さて、勾配降下法を走らせたら、どちらの谷に降りていくのでしょうか？

8

実験：初期値を変えて勾配降下法を実行してみよう

9

作戦：式は同じ、スタート地点だけ変える

本編の勾配降下法の更新式を、**そのまま**使います：

$$x_{\text{new}} = x - \alpha \cdot f'(x)$$

学習率は $\alpha = 0.1$ に固定。同じ式・同じ学習率で、**スタート地点だけ変えて** 2回実験します。

Code

```
1 def gradient_descent(f, fp, x0, alpha=0.1, tol=1e-6, max_iter=10000):
2     """勾配降下法：収束した場所・そこでの f(x)・通り道を返す"""
3     x = x0
4     history = [x]
5     for i in range(max_iter):
6         grad = fp(x)
7         if abs(grad) < tol:      # 傾きほぼゼロ → 収束
8             break
9         x = x - alpha * grad      # 傾きの逆へ、少し進む
10        history.append(x)
11    return x, f(x), history
```

実験1：\((x_0 = 0.5)\) からスタート（左の谷の近く）

手計算で追いかけてみよう

反復1

現在の位置: \((x = 0.5)\)

傾きを計算: \([f'(0.5) = 0.4 \times 0.5^3 - 3 \times 0.5^2 + 4 \times 0.5 = 0.05 - 0.75 + 2.0 = 1.3]\)

更新（傾きの逆へ、少しだけ）: \([x_{\text{new}}] = 0.5 - 0.1 \times 1.3 = 0.5 - 0.13 = 0.37\)

反復2

現在の位置: \((x = 0.37)\)

\([f'(0.37) \approx 1.090]\) \([x_{\text{new}}] = 0.37 - 0.109 = 0.261\)

反復3

現在の位置: \((x = 0.261)\)

\([f'(0.261) \approx 0.847]\) \([x_{\text{new}}] = 0.261 - 0.085 = 0.176\)

傾きの逆へ少しずつ、左の谷底 \((x = 0)\) に向かって降りています。続きはPythonに任せましょう。

❗ 実験1の結果

\((x_0 = 0.5)\) からスタート → \((x = 0)\)（左の谷の底）に収束、\((f(0) = 1)\)

Pythonで確認

▶ コードを表示

```
Result
実験1: x0 = 0.5 からスタート
=====
反復 1: x = 0.5000 → 0.3700 (f' = 1.3000)
反復 2: x = 0.3700 → 0.2610 (f' = 1.0896)
反復 3: x = 0.2610 → 0.1764 (f' = 0.8469)
... (あとは同じことの繰り返し)

★ 反復 30 回で収束: x = 0.0000, f(x) = 1.0000
```

実験2：\((x_0 = 5.5)\) からスタート（右の谷の近く）

手計算で追いかけてみよう

反復1

現在の位置: \((x = 5.5)\)

傾きを計算: \([f'(5.5) = 0.4 \times 5.5^3 - 3 \times 5.5^2 + 4 \times 5.5] [= 66.55 - 90.75 + 22.0 = -2.2]\)

更新（傾きがマイナス＝右下り坂なので、右へ進む）: \([x_{\text{new}}] = 5.5 - 0.1 \times (-2.2) = 5.5 + 0.22 = 5.72]\)

反復2

現在の位置: \((x = 5.72)\)

\([f'(5.72) \approx -0.416]\) \([x_{\text{new}}] = 5.72 + 0.042 = 5.762]\)

反復3

現在の位置: \((x = 5.762)\)

\([f'(5.762) \approx -0.037]\)

傾きはもうほぼゼロ。谷底に到着です。

❗ 実験2の結果

\((x_0 = 5.5)\) からスタート → \((x \approx 5.77)\) に収束

\((f(5.77) \approx -13.67)\) — 実験1の \((f(0) = 1)\) より **はるかに低い**

Pythonで確認

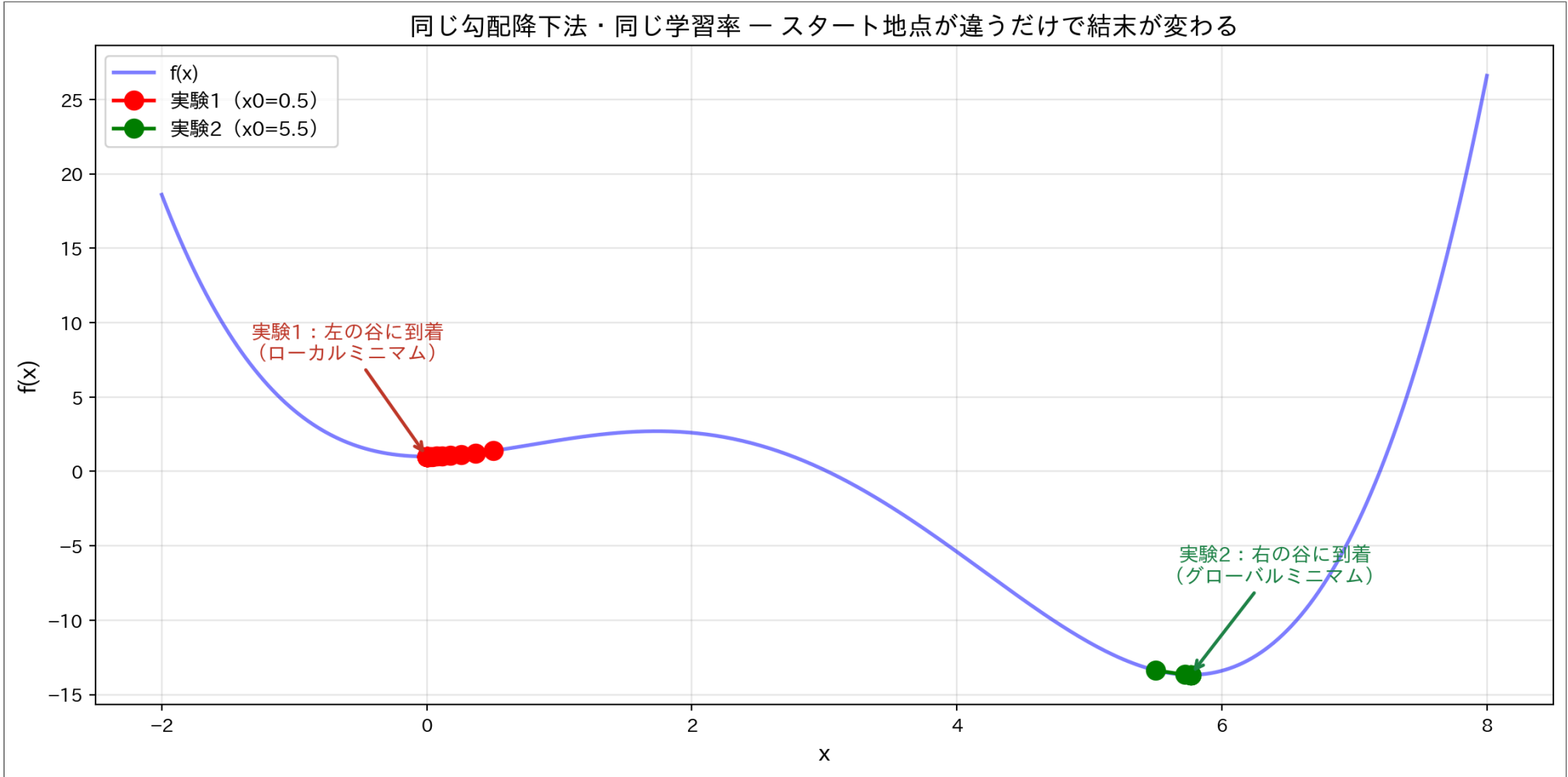
▶ コードを表示

Result

```
実験2: x0 = 5.5 からスタート
=====
反復 1: x = 5.5000 → 5.7200 (f' = -2.2000)
反復 2: x = 5.7200 → 5.7616 (f' = -0.4155)
反復 3: x = 5.7616 → 5.7653 (f' = -0.0373)

★ 反復 6 回で収束: x = 5.7656, f(x) = -13.6726
```

2つの実験を並べてみよう



⚠ たどり着いた谷が違う

まったく同じ勾配降下法を使ったのに：

	スタート地点	到着地点	f(x)の値	そこは何？
実験1	$\backslash(x_0 = 0.5\backslash)$	$\backslash(x \approx \backslash)$ 0.00	1.00	谷 (左)
実験2	$\backslash(x_0 = 5.5\backslash)$	$\backslash(x \approx \backslash)$ 5.77	-13.67	谷 (右・もっと深い)

- どちらも正真正銘の谷 ($\backslash(f''>0\backslash)$)。でも **深さがまったく違う**
- 実験1の答えは「その周辺では最適、全体では2番手」 — そして使った本人は、 **もっと深い谷の存在に気づけない**

スタート地点（初期値）が違うだけで、たどり着く谷が変わってしまう。

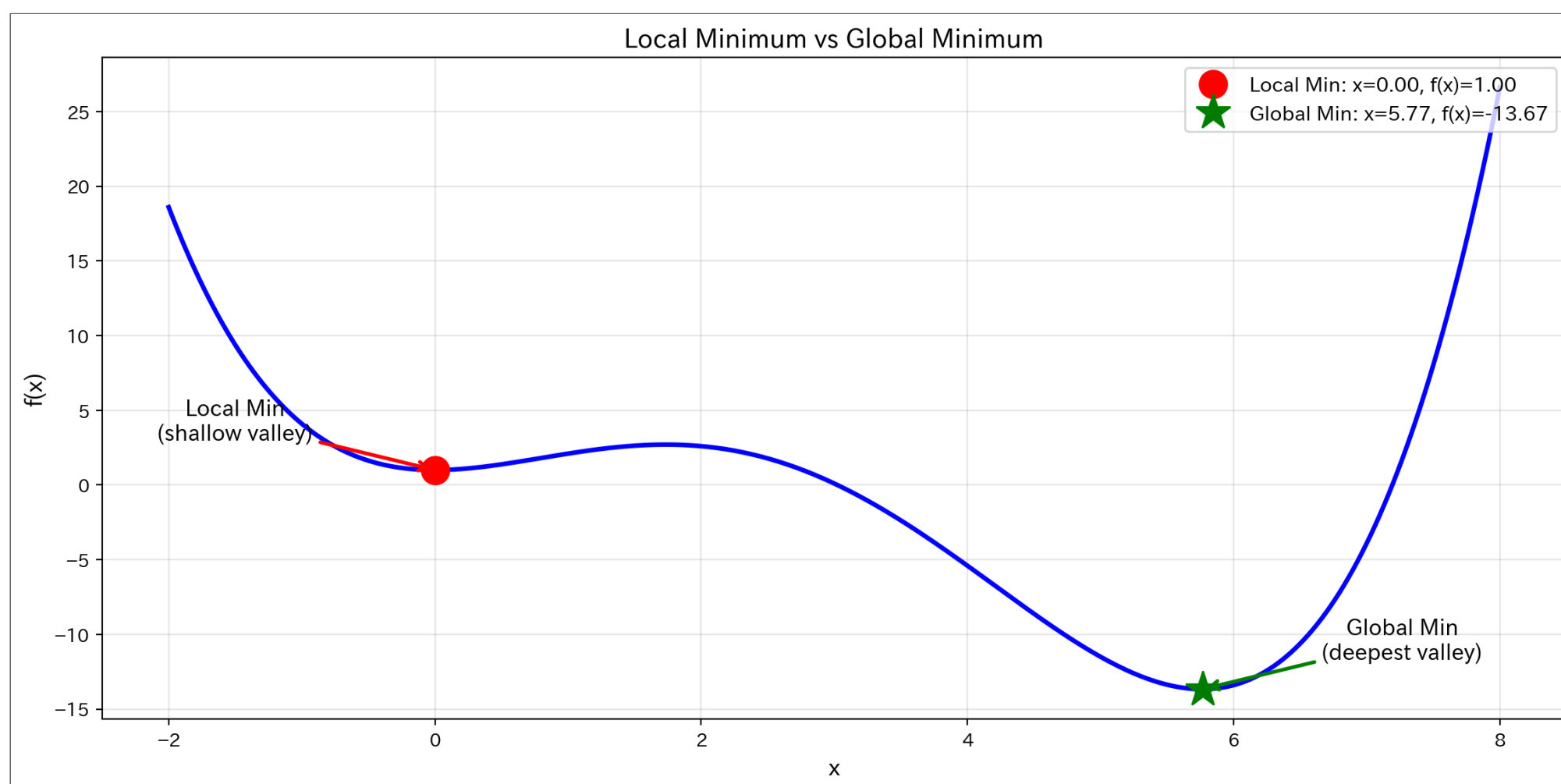
この現象に名前をつけよう

ローカルミニマムとグローバルミニマム

いま体験した現象には、ちゃんと名前がついています。

ローカルミニマム（局所的最小値, local minimum）：「その周辺では一番低い」けど、関数全体で見ると**もっと低い場所がある**点。実験1で到達した**左の谷（ $x = 0$ ）**がこれです。

グローバルミニマム（大域的最小値, global minimum）：関数全体で**本当に一番低い**点。実験2で到達した**右の谷（ $x \approx 5.77$ ）**がこれです。



💡 たとえ話：山の中で一番低い場所を探す

あなたは目隠しをして山の中にいます。足元の傾きだけを頼りに、一番低い場所を探します。

- 近くの谷に降りることはできる → **ローカルミニマム**
- でも、山の向こうにもっと深い谷があるかもしれない → **グローバルミニマム**
- 目隠しをしているので、**今いる谷が一番深いかわからない！**

Newton法も勾配降下法も、この「目隠し」と同じ状況です。

15

なぜこうなるのか：初期値と「引力圏」

16

勾配降下法は「目の前の坂」しか見ない

勾配降下法は、いまいる場所の傾きだけを見て一步を決めます。遠くにもっと深い谷があっても、**目の前の坂を下ることしかできません。**

だから、だいたいには：

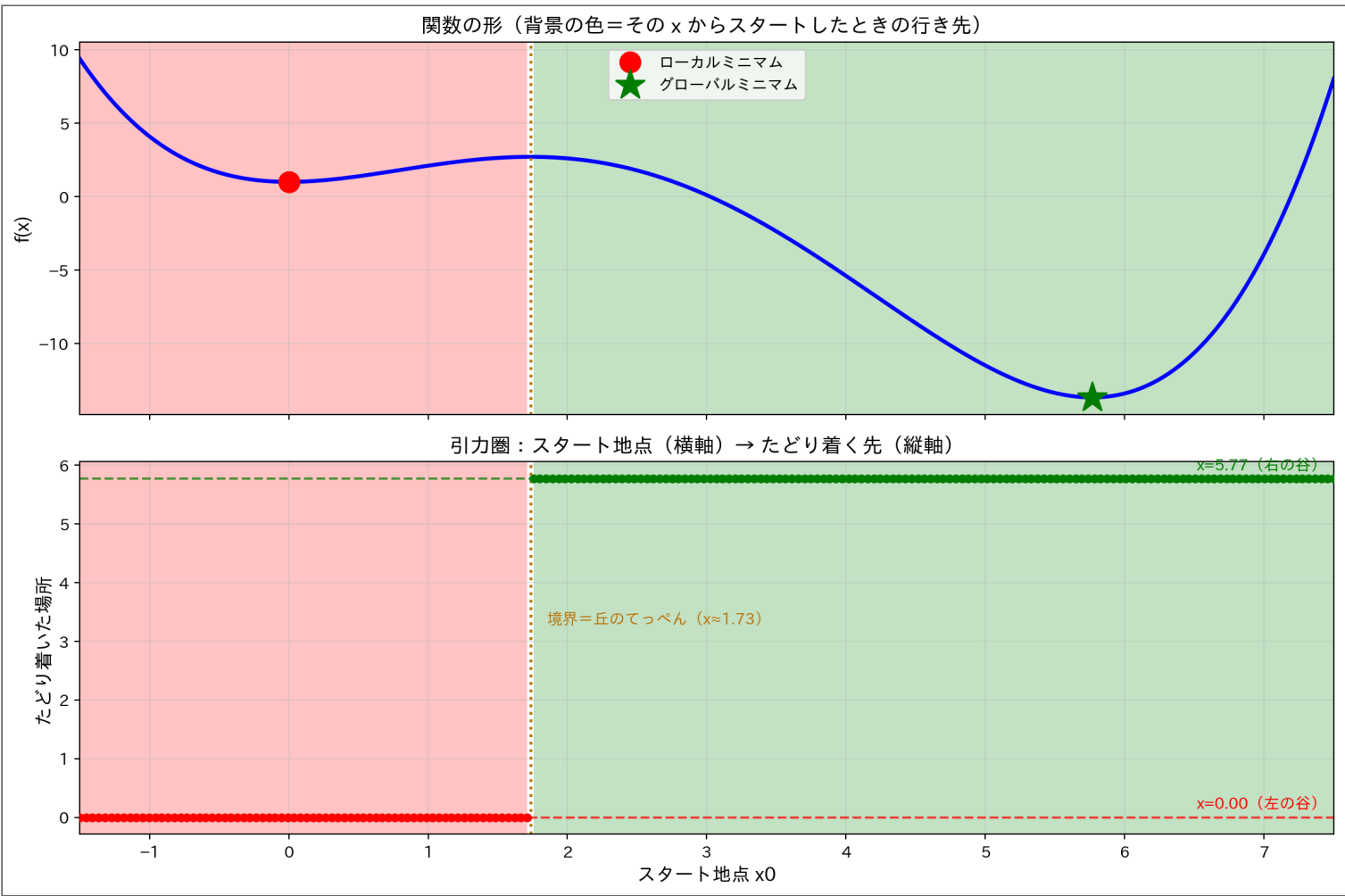
- **左の谷の近くからスタート** → 左の谷（ローカルミニマム）に落ちる（実験1）
- **右の谷の近くからスタート** → 右の谷（グローバルミニマム）に落ちる（実験2）

でも「近く」の境目はどこにあるのでしょうか。初期値を200個並べて、全部試してみましょう。

17

どの初期値が、どの谷に落ちるのか

▶ コードを表示



図の見方（下のグラフ）：

- 横軸＝**スタート地点** x_0 、縦軸＝そこから勾配降下法が**最終的にたどり着いた場所**
- 縦に1本選んで読む：「 $x_0 = 1$ から出発 → 点の高さは 0 → 左の谷に着いた」
- 点は2つの高さのどちらかに張り付く — 行き先は**左の谷（赤）**か**右の谷（緑）**の2択
- 背景の色の帯から出発すると、同じ色の谷に吸い込まれる。上下のグラフは横軸が揃えてある

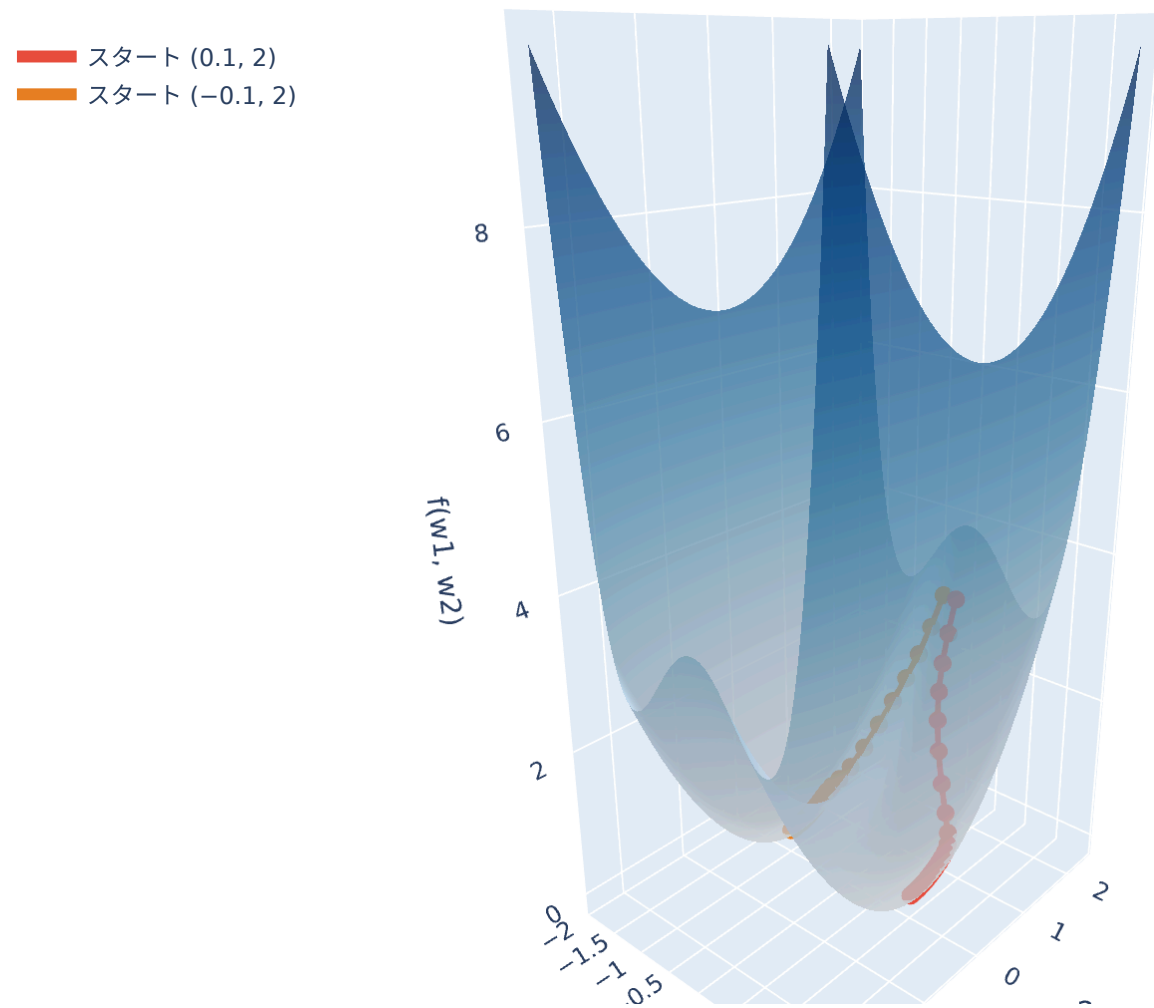
① 引力圏（Basin of Attraction）とは

各谷には**引力圏** — その範囲からスタートすると、その谷に吸い込まれる領域 — があります。

- 勾配降下法の引力圏は、**丘のてっぺん ($x \approx 1.73$) を境にしたきれいな2分割**：てっぺんより左なら左の谷、右なら右の谷
- 地図としては素直。でも裏を返せば、**答えはスタート地点を選んだ瞬間に決まっている**
- どちらの谷が深いかは、降りてみるまでわからない — **1回の実行を信用してはいけない理由**がこれ

引力圏を立体で見る

パラメータが2個になると、地形は**面**になります。谷が2つある地形 $f(w_1, w_2) = (w_1^2 - 1)^2 + 0.5w_2^2$ （谷底は $w_1 = \pm 1$ ）を、**ほんの少しだけ違う2地点**からスタートしてみます：



- スタートの差は w_1 が $(+0.1)$ か (-0.1) か**だけ**。それでも赤は右の谷、橙は左の谷に吸い込まれます
- **ドラッグで回して**、2つの谷の間の**尾根**を確認 — 尾根こそが引力圏の境界線です
- 実際の機械学習の地形は、こうした谷が無数にある高次元版。初期値しだいで別の答えに落ち着く理由がこれです

19

対策：どうすればグローバルミニмумを見つけられるか

20

打っ手はシンプル

引力圏の地図が教えてくれたこと：

- 勾配降下法は正しく動いている — ただ、**スタート地点がすべてを決めている**
- どちらの谷が深いかは、降りてみるまでわからない（目隠しの限界）

解決策：スタート地点を変えて何回もやり、一番良い結果を選ぶ！

21

方法1：手で初期値を変えて比較する

まず、3つの初期値から比較してみましょう。

$(x_0 = 0)$ からスタート

$f'(0) = 0.4 \times 0 - 3 \times 0 + 4 \times 0 = 0$

傾きがゼロなので、勾配降下法は**一步も動きません**。 $(x = 0)$ 、 $(f(0) = 1)$ で即「収束」。

$(f''(0) = 4 > 0)$ なので、ここはちゃんと谷です。ただし**谷底ど真ん中スタートだと、隣にもっと深い谷があっても永遠に気づけない**。

$(x_0 = 2)$ からスタート

$f'(2) = 0.4 \times 8 - 3 \times 4 + 4 \times 2 = 3.2 - 12 + 8 = -0.8$

傾きがマイナス（右下り坂）→ 一步は**右へ**：

$x_{\text{new}} = 2 - 0.1 \times (-0.8) = 2.08$

→ そのまま坂を下り続けて、右の谷（グローバルミニマム）へ。

$(x_0 = 6)$ からスタート

$f'(6) = 0.4 \times 216 - 3 \times 36 + 4 \times 6 = 86.4 - 108 + 24 = 2.4$

傾きがプラス（右上り坂）→ 一步は**左へ**：

$x_{\text{new}} = 6 - 0.1 \times 2.4 = 5.76$

→ こちらも右の谷へ。

3つの結果を比較

```
Code
1 # 3つの初期値から実行して比較
2 starting_points = [0, 2, 6]
3
4 print("初期値を変えて比較（勾配降下法）")
5 print("=" * 60)
6 print(f"{'x0':>4} → {'到着地点':>8} → {'f(x)':>8} → どの谷？")
7 print("-" * 60)
8
9 results_comparison = []
10 for x0 in starting_points:
11     x_final, f_final, _ = gradient_descent(f_poly, f_poly_prime, x0)
12     results_comparison.append((x0, x_final, f_final))
13
14 best_f = min(r[2] for r in results_comparison)
15 for x0, xf, fx in results_comparison:
16     where = "左の谷" if xf < 3 else "右の谷"
17     marker = " ← 最良！" if fx == best_f else ""
18     print(f"{x0:4.1f} → {xf:8.4f} → {fx:8.4f} → {where}{marker}")
```

Result

初期値を変えて比較（勾配降下法）

=====			
x0	→	到着地点	→ f(x) → どの谷？

0.0	→	0.0000	→ 1.0000 → 左の谷
2.0	→	5.7656	→ -13.6726 → 右の谷 ← 最良！
6.0	→	5.7656	→ -13.6726 → 右の谷 ← 最良！

① ポイント

3つ試すと**左右両方の谷**が出てきました。 **$f(x)$ が最小のものを採用する** — これで一番良い答えを選べます。

手間は3倍になりますが、浅い谷の答えを信じてしまうよりずっと良いです。

方法2・方法3：たくさんの初期値を自動で試す

手で3つ試すのも良いですが、コンピュータなら **20個でも100個でも** 試せます。初期値の選び方は2通りあります。

方法2：ランダムリスタート

方法3：グリッドサーチ

初期値を **ランダム** に選ぶ方法です。

▶ コードを表示

Result

ランダムリスタート：20個の初期値から試す

=====

試行 1: x0= 2.00 → x= 5.77, f(x)=-13.6726

試行 2: x0= 6.61 → x= 5.77, f(x)=-13.6726

試行 3: x0= 4.86 → x= 5.77, f(x)=-13.6726

試行 4: x0= 3.79 → x= 5.77, f(x)=-13.6726

試行 5: x0= 0.25 → x= 0.00, f(x)= 1.0000

... (残り15個)

★ 最良の結果: x0=2.00 → x=5.7656, f(x)=-13.6726

ランダムではなく、 **等間隔に並べる** 方法もあります。

▶ コードを表示

Result

グリッドサーチ：等間隔の15個の初期値

=====

x0 → 到着地点 → f(x)

-1.00 → -0.0000 → 1.0000

-0.43 → -0.0000 → 1.0000

0.14 → 0.0000 → 1.0000

0.71 → 0.0000 → 1.0000

1.29 → 0.0000 → 1.0000

1.86 → 5.7656 → -13.6726

2.43 → 5.7656 → -13.6726

3.00 → 5.7656 → -13.6726

3.57 → 5.7656 → -13.6726

4.14 → 5.7656 → -13.6726

4.71 → 5.7656 → -13.6726

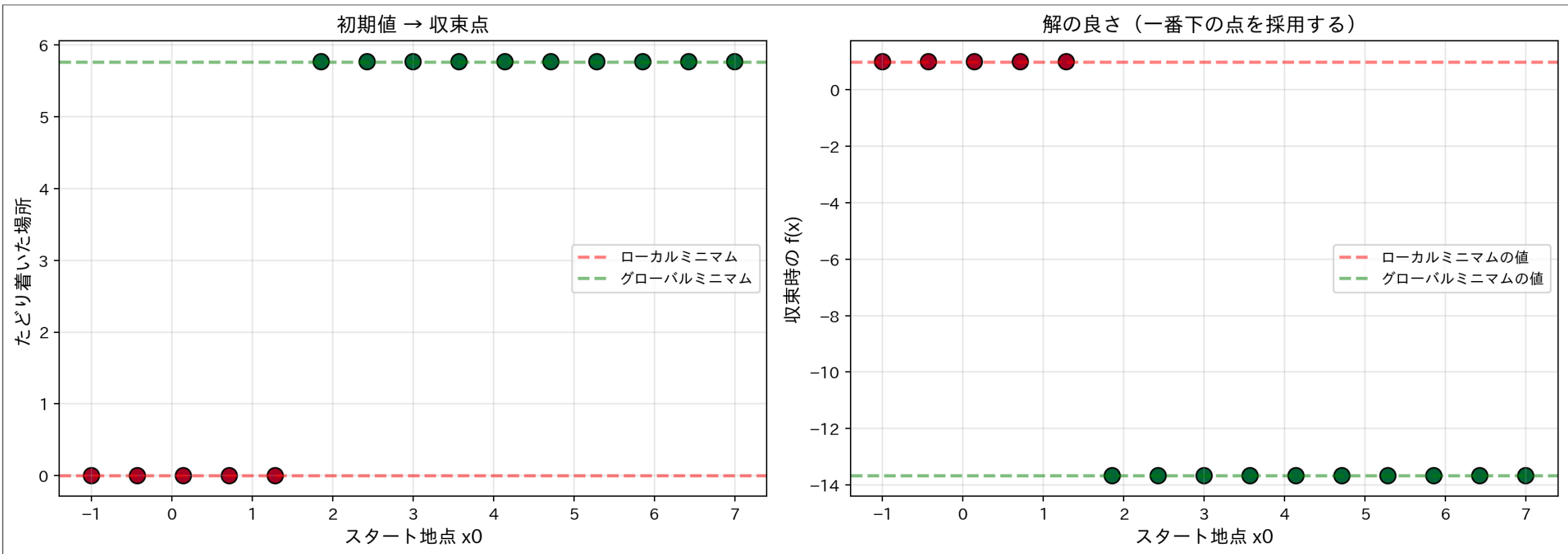
5.29 → 5.7656 → -13.6726

5.86 → 5.7656 → -13.6726

6.43 → 5.7656 → -13.6726

7.00 → 5.7656 → -13.6726

★ 最良: x0=3.57 → x=5.7656, f(x)=-13.6726



💡 ランダムリスタート vs グリッドサーチ

方法	やり方	長所	短所
ランダムリスタート	初期値をランダムに選ぶ	簡単、偏りが少ない	運に左右される
グリッドサーチ	初期値を等間隔に並べる	漏れが少ない	数が多いと大変

どちらも「何回も試して一番良いのを選ぶ」という同じ考え方です。

この作戦、実はもう使っている

💡 k-means の n_init=10 の正体

クラスタリングの回で、k-means が**クラスタ中心の初期値**しだいで変な分け方に落ち着くのを見ました。あのとき出てきた `KMeans(..., n_init=10)` は「初期値を変えて10回学習し、inertia が最小の結果を採用」 — つまり **ランダムリスタート** そのものです。

「初期値を変えて何回も試し、一番良いのを選ぶ」 — この作戦の一族をまとめて **マルチスタート (multi-start)** と呼びます。

- k-means の `n_init` はマルチスタート
- ニューラルネットワークで「乱数シードを変えて学習し直す」のもマルチスタート
- 名前は違っても、やっていることは今日の「方法1〜3」と同じ

実践チェックリスト

最適化の4ステップ

最適化問題に取り組むときは、以下の手順を守りましょう。

① ステップ1：関数を可視化する

まずグラフを描いて、谷がいくつあるか確認する。谷が複数ありそうなら、ローカルミニマムに注意！

① ステップ2：複数の初期値を用意する

最低でも3～5個。余裕があれば10個以上。グリッドサーチかランダムリスタートで用意する。

① ステップ3：全部実行して、結果を比較する

すべての初期値から最適化（勾配降下法など、手法は何でもよい）を実行し、 $f(x)$ が**一番小さい**結果を選ぶ。

① ステップ4：2次微分で確認する

収束した点で $f''(x) > 0$ なら谷（最小値）、 $f''(x) < 0$ なら山なので捨てる。坂を下ることしかしない勾配降下法ではまず山に止まらないが、確認は1行 — **保険として常に入れておく**（このあとの発展編で見るNewton法では必須）。

実装テンプレート

上の4ステップをまとめたコードです。コピーして使えます。

Code

```
1 def robust_optimization(f, fp, fpp, x0_list, alpha=0.1, max_iter=10000):
2     """
3     複数の初期値を使ったロバストな最適化（勾配降下法版）
4
5     Parameters:
6     -----
7     f, fp, fpp : functions
8         目的関数とその1次微分、2次微分（2次微分は谷の検品用）
9     x0_list : list
10        初期値のリスト
11     alpha : float
12        学習率
13     max_iter : int
14        最大反復数
15
16     Returns:
17     -----
18     best_x : float
19        見つかった最良解
20     all_results : list
21        比較用のすべての結果
22     """
23     all_results = []
24
25     for x0 in x0_list:
26         x = x0
27
28         for i in range(max_iter):
29             grad = fp(x)
30             if abs(grad) < 1e-6:          # 傾きほぼゼロ → 収束
31                 break
32             x = x - alpha * grad         # 傾きの逆へ、少し進む
33
34         all_results.append({
35             "x0": x0,
36             "x_final": x,
37             "f_final": f(x),
38             "is_minimum": fpp(x) > 0    # ステップ4：谷の検品
39         })
40
41     # 谷（最小値）だけを残して、一番良いのを選ぶ
42     minima = [r for r in all_results if r["is_minimum"]]
43     if minima:
44         best_result = min(minima, key=lambda r: r["f_final"])
45     else:
46         best_result = min(all_results, key=lambda r: r["f_final"])
47
48     return best_result["x_final"], all_results
49
50 # 使用例：5つの初期値から試す
51 initial_values = [-1, 0, 2, 4, 6]
52 best_x, all_results = robust_optimization(f_poly, f_poly_prime, f_poly_double_prime,
53                                           initial_values)
54
55 print("すべての結果:")
56 print(f"{'x0':>4} → {'到着地点':>8} → {'f(x)':>8} → {'谷?':>4}")
57 print("-" * 50)
58 for r in all_results:
59     valley = "はい" if r["is_minimum"] else "いいえ"
```

```
60     marker = " ← 最良!" if r["x_final"] == best_x else ""
61     print(f"{r['x0']:4.1f} → {r['x_final']:8.4f} → {r['f_final']:8.4f} → {valley}{marker}")
62
63 print(f"\n★ 最良解: x = {best_x:.4f}, f(x) = {f_poly(best_x):.4f}")
```

Result

すべての結果:

x0	→	到着地点	→	f(x)	→	谷?

-1.0	→	-0.0000	→	1.0000	→	はい
0.0	→	0.0000	→	1.0000	→	はい
2.0	→	5.7656	→	-13.6726	→	はい ← 最良!
4.0	→	5.7656	→	-13.6726	→	はい
6.0	→	5.7656	→	-13.6726	→	はい

★ 最良解: x = 5.7656, f(x) = -13.6726

発展：Newton法だと、もっと変なことが起こる

Newton法でやり直すと

Newton法（参考資料） は、いまいる場所に放物線を当てはめて、 **その頂点へ一気にジャンプ** する方法でした：

$$x_{\text{new}} = x - \frac{f'(x)}{f''(x)}$$

実験1・実験2と同じ初期値で、Newton法を走らせてみます。

▶ コードを表示

Result

素のNewton法 vs 勾配降下法（反復回数）

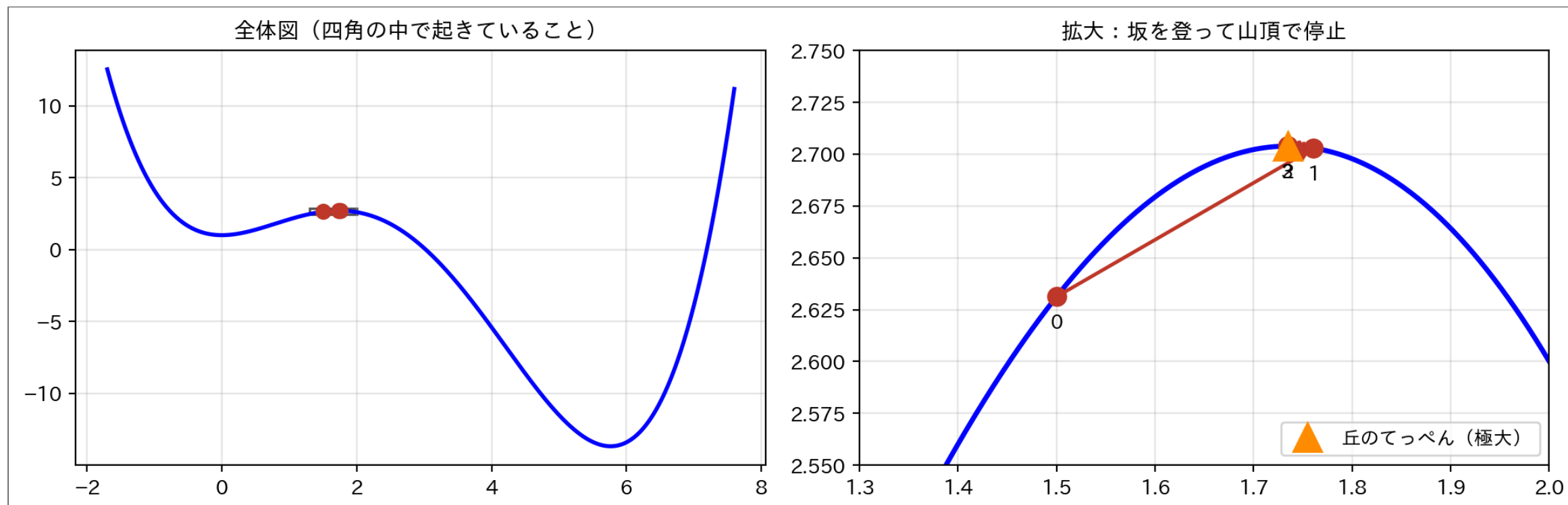
=====

x0 = 0.5:	Newton法	5 回	/	勾配降下法	30 回	→	x = -0.0000
x0 = 5.5:	Newton法	4 回	/	勾配降下法	6 回	→	x = 5.7656

- 行き先は勾配降下法と同じ2択（左の谷・右の谷）。しかも **一桁の反復回数** — 圧倒的に速い
- 「じゃあ全部Newton法でいいのでは？」 — そうはいかないんです。スタート地点をもう少し意地悪にしてみます

実験3：丘の中腹 $(x_0 = 1.5)$ から始めると...

- 同じ更新式を回すと： $(1.5 \rightarrow 1.761 \rightarrow 1.735 \rightarrow)$ 停止 ($f' \approx 0$)
- 左に下り坂があるのに、**坂を登って丘のてっぺんで止まった**



なぜ**登る**のか：

- 一步の向きは $(-f'/f'')$ 。ここで $(f'(1.5) = +0.6)$ (右上り坂)、 $(f''(1.5) = -2.3)$ (**上に凸**)
- 符号が打ち消し合って、一步は $(+0.26)$ — **傾きと同じ向き、つまり登る向き** になる
- Newton法が探しているのは「 $(f' = 0)$ の点」だけ。そこが山か谷かは**見ていない**
- 停止条件 ($(|f'|)$ がほぼゼロ) は山頂でも満たされる → **山で本当に止まる**

💡 (f'') チェックは「止まった後の検品」

収束したら $(f''(x))$ を確認：**正なら谷、負なら山 — 山なら結果を捨てる**。山か谷かの区別は**できる**が、それは止まってからの話。走っている最中のNewton法には、登りを避けるブレーキがついていない。（実は後付けできる — 引力圏とカタパルトを見たあとで「修理」する）

30

Newton法は「近くの傾きゼロ」に落ちる

Newton法が探しているのは「傾きゼロの点」。そこが谷か山頂かは**見ていません**。

- 谷の近くから → その谷に落ちる (さっきの再実験)
- 丘の上から → **山頂に登って止まる** (実験3)

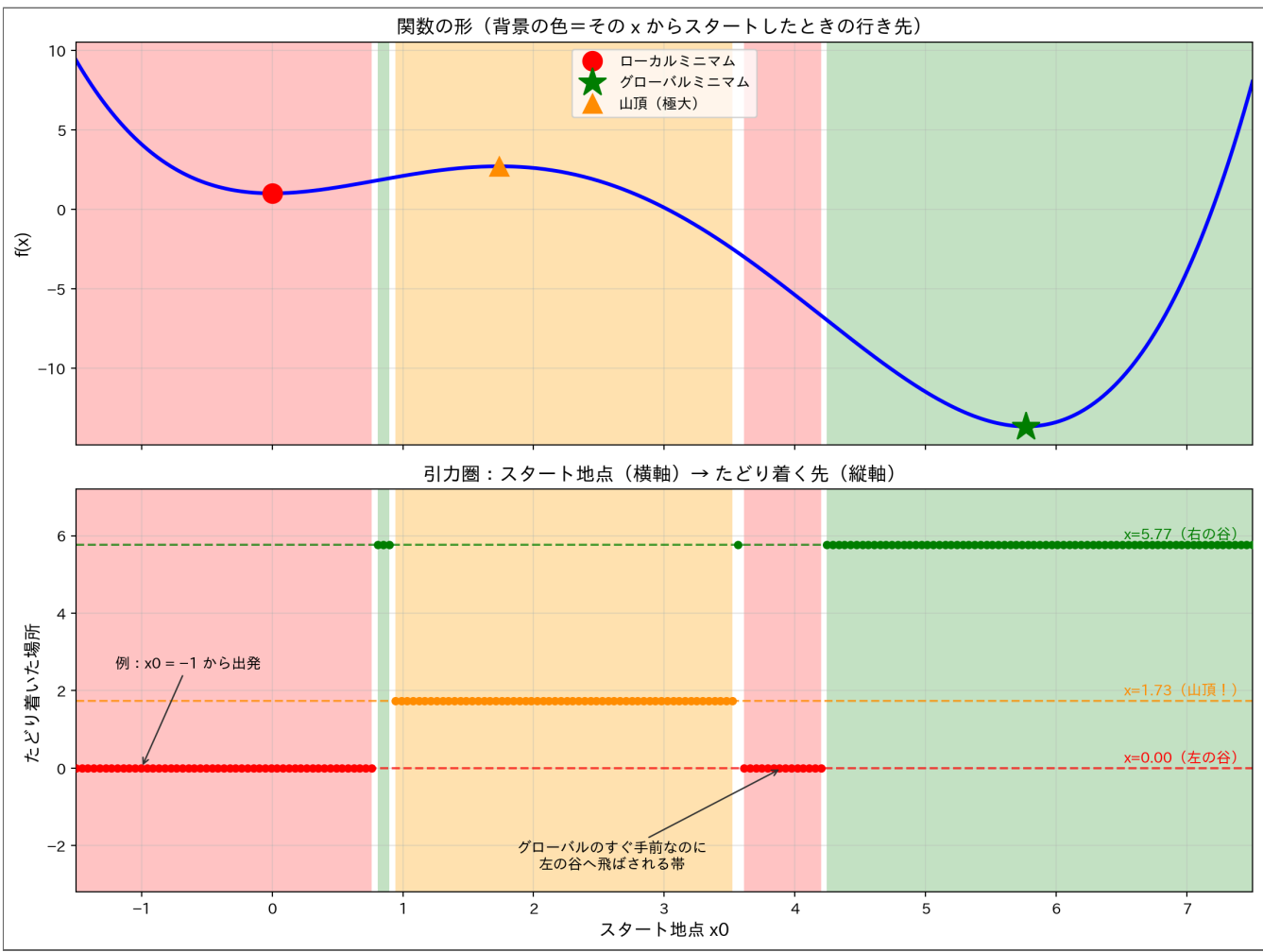
被害はこれだけなのでしょうか。勾配降下法の時と同じように、初期値200個で引力圏の地図を描いてみましょう。

31

Newton法の引力圏を可視化しよう

勾配降下法では「丘のてっぺんを境にきれいな2分割」でした。Newton法ではどうなるでしょうか。

▶ コードを表示



図の見方（下のグラフ）：

- 横軸＝**スタート地点** x_0 、縦軸＝そこから Newton法が**最終的にたどり着いた場所**
- 縦に1本選んで読む：「 $x_0 = -1$ から出発 → 点の高さは 0 → 左の谷に着いた」
- 点は3つの高さのどれかに張り付く — 行き先は **左の谷（赤）・右の谷（緑）・山頂（橙）** の3択
- 背景の色の帯が**引力圏**：同じ色の帯から出発すると、同じ場所に吸い込まれる
- 上下のグラフは横軸が揃えてある — 上の山谷と下の帯を見比べられる

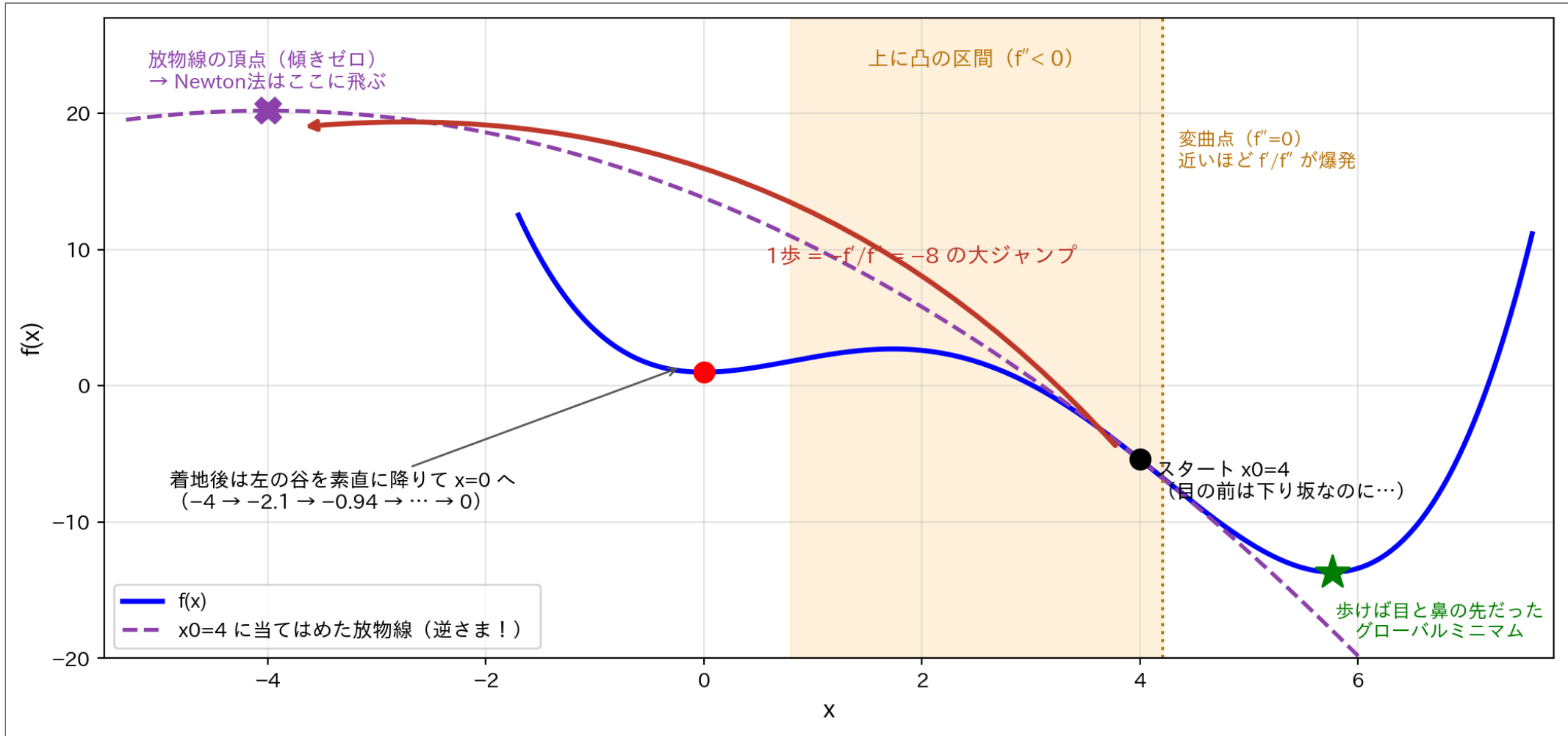
① Newton法の引力圏は、素直じゃない

勾配降下法の「きれいな2分割」と見比べると、意地悪な結果になりました：

- 赤い帯 → ローカルミニマム、緑の帯 → グローバルミニマム、ここまでは予想どおり
- **橙の帯 → 山のとっぺん（極大）に到着して止まる**。Newton法が探しているのは「傾きゼロの場所」なので、谷と山頂を **区別できない**
- 帯はきれいな2分割ではない：**グローバルミニマムのすぐ手前（ $x_0 \approx 3.6 \sim 4.2$ ）から出発しても、左の谷へ飛ばされる**
- 初期値が少し変わるだけで行き先がガラッと変わる — 1回の実行を信用してはいけない理由がこれ

なぜ「すぐ手前の帯」が左へ飛ぶのか

$x_0 = 4.0$ から始めたときの**最初の1歩**を解剖してみます。



- Newton法の1歩は「いまの場所に**放物線を当てはめて、その頂点へジャンプ**」($x - f'/f''$) の意味)
- ところが $x_0=4$ は**上に凸の区間** (2つの変曲点 $x \approx 0.79 \sim 4.21$ の間) の右端 — 当てはめた放物線が**逆さま**になり、頂点は左の遠く ($x=-4$) にある
- しかも変曲点の近くでは $f'' \approx 0$ なので、割り算 f'/f'' が**爆発**して飛距離が伸びる
- 着地した $x=-4$ は左の谷の素直な (下に凸の) 区間 → あとはおとなしく $x=0$ に収束
- 細い緑の帯 ($x_0 \approx 0.85$) は同じ現象の**逆向き**：左の変曲点のすぐ内側から $+6.3$ ジャンプして右の谷へ

❗ 重要

大ジャンプできるのがNewton法の強みで、放物線近似が嘘をつく場所では同じ大ジャンプが暴発する。 勾配降下法なら「傾きの逆に少しずつ」なので、 $x_0=4$ からは素直に右の谷へ降りる (前半の引力圏の地図のとおり)。

— では、この壊れ方は直せないのか？ **直せます**。次で修理しましょう。

修理する：ワープの前に「検品」を入れる

故障の原因は「無条件ワープ」

ここまでに見た故障は2種類：

- **山頂で止まる**（実験3） — 上に凸の場所では、一步が**登る向き**になる
- **反対側へ吹っ飛ぶ**（カタパルト） — 変曲点の近くでは、一步が**大きすぎる**

原因はどちらも同じ。Newton法が、当てはめた放物線の頂点へ**検査なしでワープ**していることです。

💡 ヒント

修理方針：提案された一步を、**2つの検品**に通してから踏む。

35

検査1：その方向、本当に「下り」？

- Newton法の一步 $d = -f'/f''$ は**提案**として扱う
- 踏む前にチェック： $f' \cdot d \geq 0$ なら、提案は**傾きと同じ向き＝登り**なので**却下**
- 却下したら、確実に下る**最急降下方向** $d = -f'$ に差し替える
- 登り提案が出るのは、ちょうど $f'' < 0$ （上に凸）の場所 — 実験3とカタパルトの現場そのもの

実験3の現場検証： $x = 1.5$ では提案 $d = +0.26$ 。 $f'' \cdot d = 0.6 \times 0.26 > 0$
→ 登りなので却下 → $d = -f' = -0.6$ （左の下りへ）

36

検査2：その一步、実測で下がってる？

- 向きが正しくても、**歩幅が大きすぎる**と谷を飛び越えて高い場所に着くことがある
- そこで、踏む前に着地点の**本物の f** を**実測**する：
 - 十分下がっていれば採用
 - 下がっていなければ**歩幅を半分**にしてやり直し（バックトラッキング）
- 放物線の「予言」ではなく、**実測**で確認してからしか進まない

37

修理版Newton法（検査1・2入り）

Code

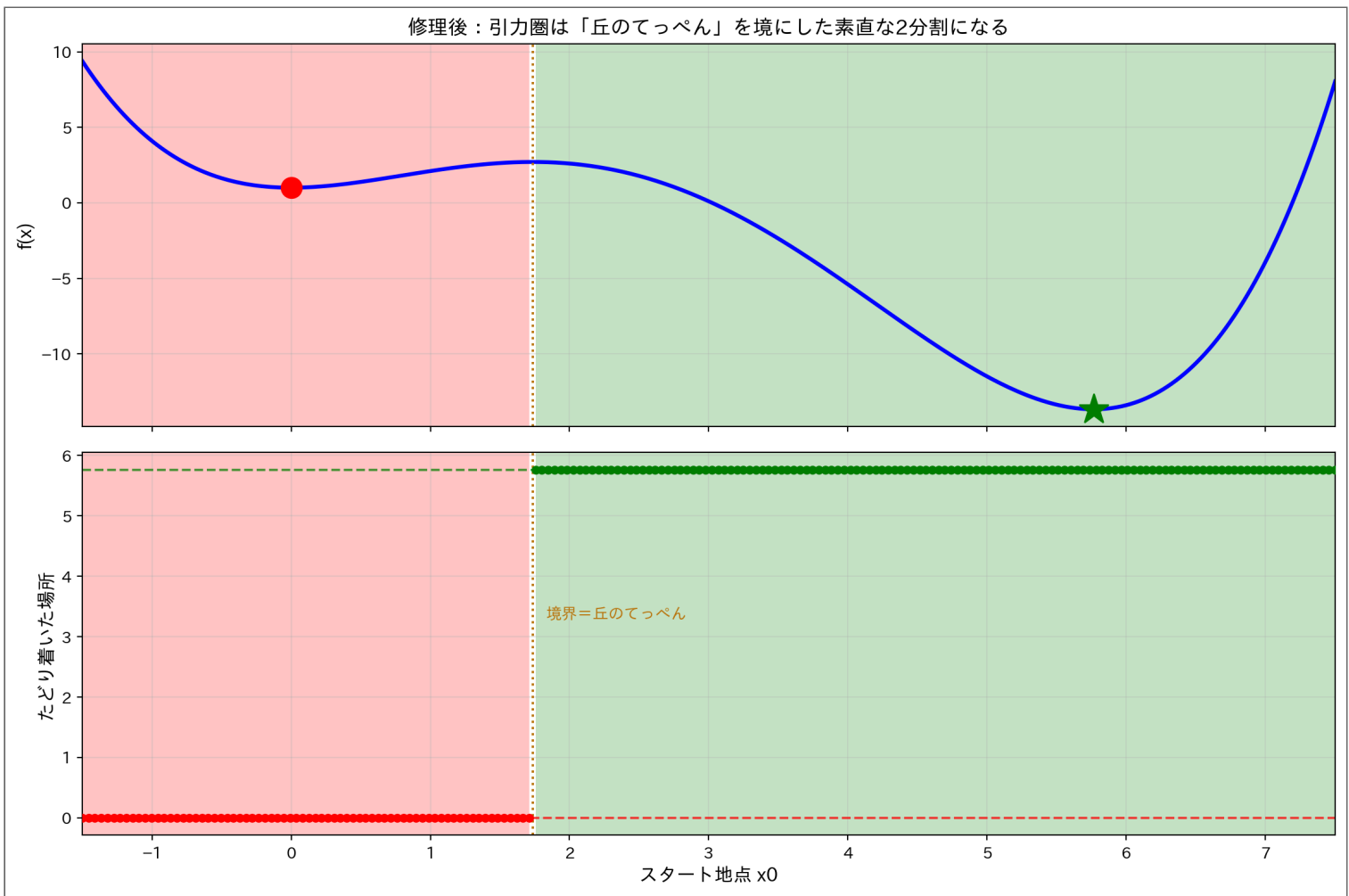
```
1 def newton_safe(f, fp, fpp, x0, max_iter=100):
2     """Newton法+検査1（降下方向）+検査2（実測バックトラッキング）"""
3     x = x0
4     for _ in range(max_iter):
5         g = fp(x)
6         if abs(g) < 1e-8:                                # 傾きほぼゼロ → 停止
7             break
8         h = fpp(x)
9         d = -g / h if h != 0 else -g                    # Newton法の「提案」
10        if g * d >= 0:                                    # 検査1：登りなら却下して
11            d = -g                                        # 最急降下に差し替え
12        alpha = 1.0
13        while f(x + alpha * d) > f(x) - 1e-4 * alpha * abs(g * d):
14            alpha /= 2                                    # 検査2：実測で下がるまで歩幅半分
15        x = x + alpha * d
16    return x, f(x)
17
18 # 故障していた2つの初期値で試運転
19 for x0 in [1.5, 4.0]:
20     x_fin, f_fin = newton_safe(f_poly, f_poly_prime, f_poly_double_prime, x0)
21     print(f"x0 = {x0} → x = {x_fin:.4f}, f(x) = {f_fin:.4f}")
```

Result

x0 = 1.5 → x = -0.0000, f(x) = 1.0000
x0 = 4.0 → x = 5.7656, f(x) = -13.6726

- 山頂に登っていた $(x_0 = 1.5)$ は**左の谷**へ降りた
- カタパルトだった $(x_0 = 4)$ は、なんと**右の谷（グローバルミニマム）**へ — 暴発も消えた

修理後の引力圏



- 山頂の帯（橙）も、飛ばされる帯も消えた — てっぺんより左なら左の谷、右なら右の谷
- 検査1・2で直るのは、ここまで

ライブラリは最初から修理済み

- scipy などの実用ソルバーは、この種の検品を **毎歩** 組み込み済み：
 - **BFGS**：常に「下り向きの提案」しか出ないように作った近似 $\nabla^2 f$ を使う
 - **Newton-CG**：負の曲率（上に凸）を検出した瞬間に計算を打ち切り、勾配方向へ切り替え
 - **信頼領域法**：放物線を「信じる半径」の中でだけ使う
- 実測： $x_0 = 1.5$ から **scipy.optimize.minimize** を呼ぶと、BFGS / Newton-CG / trust-exact のすべてが山頂に登らず $x=0$ の谷に降りる
- 流儀は違ってても思想は同じ： **f が下がらない一歩は踏まない**

① ノート

授業で書いた素のNewton法が「欠陥品」なわけではない。教科書のNewton法はもともと「 $f'=0$ 」を解く」方法で、**最小化に使うための安全装置**は別売り — ライブラリにはそれが標準装備されている、ということ。

40

発展編のまとめ：修理して直るのは「事故」だけ

- **山頂で止まる・反対側へ吹っ飛ぶ**は、検査1・2で直せる — ライブラリは標準装備済み
- でも修理後も、行き先は相変わらず**2つ**ある — どの谷に落ちるかは初期値しだいのまま
- これはNewton法の故障ではなく、**目隠しで坂を下ること自体の限界**（目の前の谷しか見えない）
- だから、マルチスタート（方法1～3）は**どんな手法を使っても必須**

41

まとめ

42

① ノート

1. **谷が複数ある関数では、答えが初期値に依存する**
 - 勾配降下法は目の前の坂を下ることしかできない — 近くの谷に落ちる
 - どの初期値がどの谷に行くかの地図が**引力圏**（境界は丘のてっぺん）
2. **ローカルミニマムとグローバルミニマム**
 - ローカルミニマム = 周辺では一番低いが、全体では最低ではない
 - グローバルミニマム = 全体で本当に一番低い点
3. **対策はマルチスタート：複数の初期値から試して、一番良い結果を選ぶ**
 - ランダムリスタートやグリッドサーチで初期値を用意する
 - k-means の **n_init=10** はこの作戦そのもの
 - 2次微分の確認 ($f'' > 0$) なら谷）は保険として常に入れる
4. **発展：Newton法は速いが、「山頂停止」「カタパルト」という事故も起こす**
 - 検査1（下り向きか）・検査2（実測で下がったか）で事故は直せる — ライブラリは修理済み
 - それでも「どの谷か」の2択は残る — マルチスタートに代わりはない

💡 この話、ニューラルネットワークの本番でまた会う

ニューラルネットワークの損失関数は、パラメータが何万個もある **谷だらけの地形** です。同じデータで学習し直すたびに結果が微妙に違うのは、初期値（乱数）しだいで **別の谷に落ちている** から。「何回か学習して一番良いのを選ぶ」は、深層学習でも現役の作戦です。

43

練習問題

以下の問題をPythonコードで解け（導関数だけ手計算）。

問題1：新しい地形でマルチスタート

関数 $g(x) = x^4 - 8x^2 + 3x$ について：

1. $x \in [-3.5, 3.5]$ の範囲でグラフを描き、谷の数を確認せよ
2. 導関数 $g'(x)$ を手計算で求めよ
3. 初期値 $x_0 = -3, 0, 3$ の3つから勾配降下法を実行せよ（学習率 $\alpha = 0.01$ ）、到着地点と $g(x)$ を表示
4. グローバルミニマムはどちらの谷か、 $g(x)$ の値で判定せよ

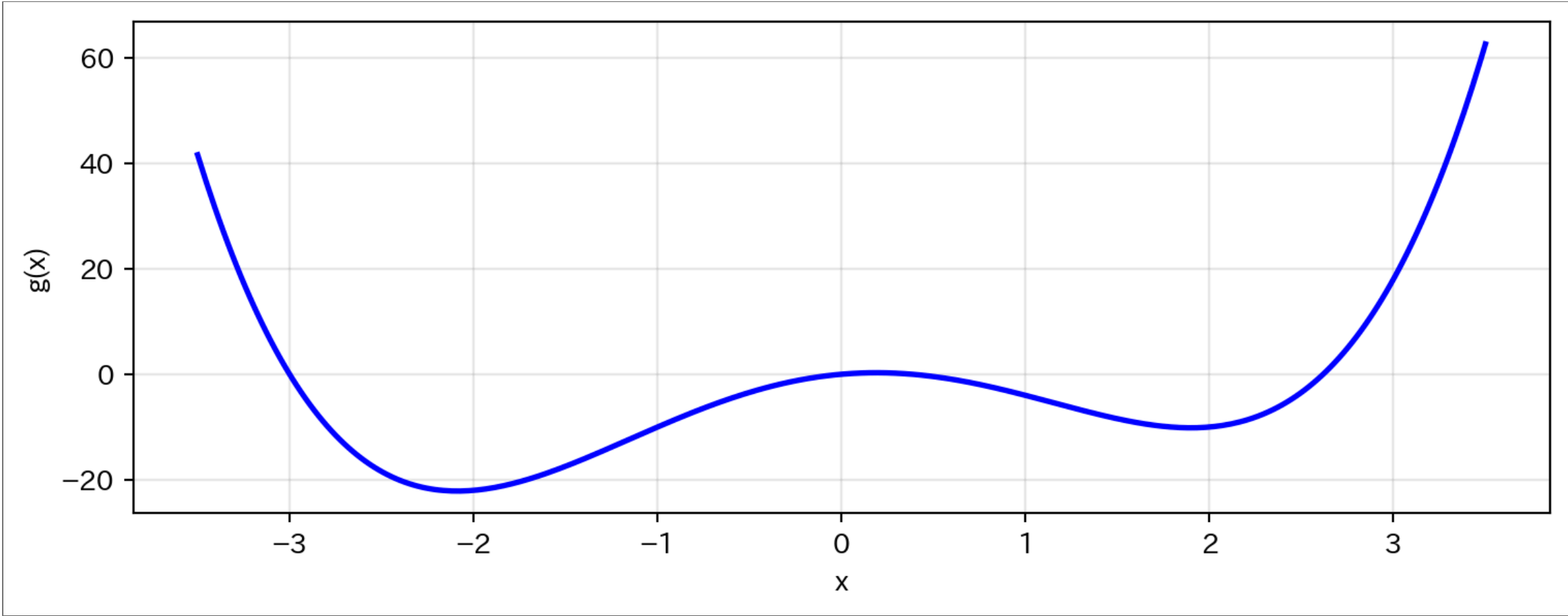
ヒント：今回の資料の関数とは、**深い谷の位置が違う** かもしれない。

44

問題1の解答例

導関数（手計算）： $[g'(x) = 4x^3 - 16x + 3]$

▶ 解答例を見る



Result		
x_0	→ 到着地点	→ $g(x)$
-3.0	→ -2.0879	→ -22.1346
0.0	→ -2.0879	→ -22.1346
3.0	→ 1.8987	→ -10.1479
★ グローバルミニマム: $x = -2.0879, g(x) = -22.1346$		

- $(x_0 = -3, 0) \rightarrow$ **左の谷** ($x \approx -2.1$)、 $(x_0 = 3) \rightarrow$ 右の谷 ($x \approx 1.9$)
- $(x_0 = 0)$ は傾き $(g'(0) = 3)$ (右上り坂) なので**左へ**転がり落ちる
- 左の谷が $(g \approx -22)$ で**グローバルミニマム** — 今回の資料の関数とは逆に、**今度は左が正解**
- どちらの谷が深いかは関数ごとに違う。だから毎回、複数の初期値で確かめる